Network Packet Analyzer (Mini Wireshark)

Complete Technical Report with Full Code and Line-by-Line Explanation

Ansh Prem

Contents

1 Introduction **3**
1.1 Why this project matters . . . 3

2 Core Features **3**

3 System Architecture **3**
3.1 High-level pipeline . . . 3
3.2 Concurrency model (why it is designed this way) . . . 4

4 Folder and File Structure **4**
4.1 Folder tree . . . 4
4.2 What each folder contains . . . 5

5 How To Run (Quick + Full) **5**
5.1 Install dependencies (Linux/Ubuntu/WSL) . . . 5
5.2 Build . . . 5
5.3 Run (examples) . . . 5
5.4 Generate traffic to test . . . 5
5.5 Open PCAP in Wireshark . . . 5

6 FULL CODE WITH LINE-BY-LINE EXPLANATION **6**
6.1 CMakeLists.txt (Build System) . . . 6
6.2 include/npa/config.hpp (CLI Configuration) . . . 6
6.3 src/config.cpp (CLI Parsing with Boost.Program_options) . . . 7
6.4 include/npa/packet.hpp (PacketInfo Model) . . . 9
6.5 include/npa/util.hpp and src/util.cpp (Byte Helpers) . . . 10
6.6 include/npa/pcap$_{c}apture.hppandsrc/pcap_{c}apture.cpp(CaptureLayer)$ . . . 11
6.7 include/npa/pcap$_{w}riter.hppandsrc/pcap_{w}riter.cpp(PCAPExport)$ . . . 15
6.8 include/npa/parser.hpp and src/parser.cpp (Protocol Parsing + HTTP Host Extraction) . . . 17
6.9 include/npa/analytics.hpp and src/analytics.cpp (Flows, Talkers, HTTP Hosts) . . . 22
6.10 include/npa/stats.hpp and src/stats.cpp (Counters + Reporting) . . . 27
6.11 src/main.cpp (Orchestration, Threads, Timers, Pipeline) . . . 31

7 Extra Diagrams for Interviews **35**
7.1 Call Graph . . . 35
7.2 Dependency Map . . . 35

8 Notes and Limitations (Honest Engineering) 35

9 Future Work (Resume-Heavy Extensions) 36

10 Conclusion 36

1 Introduction

This document is a complete explanation report for a **Network Packet Analyzer** implemented in **C++17**. The tool captures packets from a real network interface, parses protocol headers, performs traffic analytics (Top Talkers, Flow Tracking, HTTP Host extraction), and optionally saves captured traffic to a Wireshark-compatible **.pcap** file.

1.1 Why this project matters

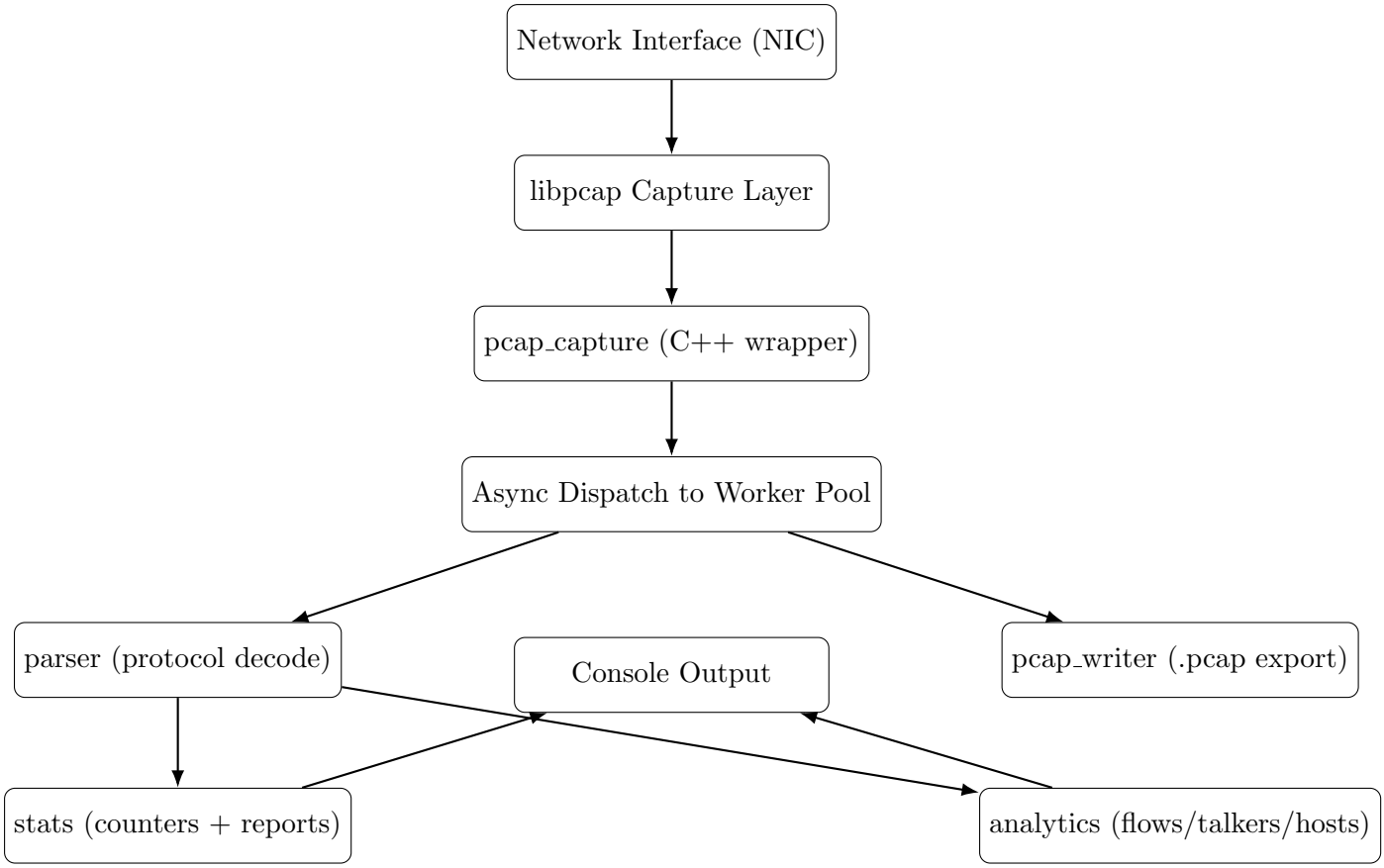
Networks are the backbone of modern systems. Debugging latency, diagnosing failures, monitoring bandwidth usage, and doing security analysis all require visibility into network traffic. Tools like Wireshark are powerful but complex. This project demonstrates the **core engineering concepts** behind such tools: packet capture, protocol parsing, concurrent processing, real-time analytics, and exportable artifacts.

2 Core Features

- **Live packet capture** using libpcap (Linux/macOS) / Npcap (Windows).
- **Protocol parsing**: Ethernet → IPv4 → TCP/UDP.
- **Real-time statistics**: packet counts, bytes, protocol split.
- **Top Talkers**: top source and destination IPs by bytes.
- **Flow tracking**: 5-tuple (canonicalized) conversation tracking.
- **HTTP Host extraction**: parses plaintext HTTP requests to extract Host header.
- **PCAP export**: saves packets to `capture.pcap` readable in Wireshark.
- **Async + multithreaded design**: capture thread + Boost.Asio worker pool.

3 System Architecture

3.1 High-level pipeline



3.2 Concurrency model (why it is designed this way)

- **Capture thread**: continuously pulls packets from libpcap, ensuring we do not miss packets.
- **Worker pool (Boost.Asio)**: CPU-heavy parsing and analytics are executed in parallel.
- **Reason**: capture should never block on parsing; otherwise packet loss increases.

4 Folder and File Structure

4.1 Folder tree

```
network-packet-analyzer/
 include/npa/
    analytics.hpp
    config.hpp
    packet.hpp
    parser.hpp
    pcap_capture.hpp
    pcap_writer.hpp
    stats.hpp
    util.hpp
 src/
    analytics.cpp
    config.cpp
```

```
    main.cpp
    parser.cpp
    pcap_capture.cpp
    pcap_writer.cpp
    stats.cpp
    util.cpp
 CMakeLists.txt
 README.md
```

4.2 What each folder contains

- **include/npa/**: Header files (.hpp) that define interfaces, structs, and class APIs.
- **src/**: Implementation files (.cpp) that implement the actual logic.
- **CMakeLists.txt**: Build instructions.
- **README.md**: User documentation (how to build/run + feature summary).

5 How To Run (Quick + Full)

5.1 Install dependencies (Linux/Ubuntu/WSL)

```
sudo apt update
sudo apt install build-essential cmake libboost-all-dev libpcap-dev
```

5.2 Build

```
mkdir build
cd build
cmake ..
make -j
```

5.3 Run (examples)

```
sudo ./npa --iface eth0
sudo ./npa --iface eth0 --filter "tcp" -v
sudo ./npa --iface eth0 --pcap-out capture.pcap
sudo ./npa --iface eth0 --top-talkers 5 --top-flows 5 --top-hosts 5 -v
```

5.4 Generate traffic to test

```
curl http://example.com
curl https://google.com
ping google.com
```

5.5 Open PCAP in Wireshark

```
wireshark capture.pcap
```

6 FULL CODE WITH LINE-BY-LINE EXPLANATION

6.1 CMakeLists.txt (Build System)

Purpose: Defines how to compile and link the project with Boost and libpcap.

Listing 1: CMakeLists.txt (annotated)

```
cmake_minimum_required(VERSION 3.16)                    # Require a modern CMake version for consistent behavior
project(npa LANGUAGES CXX)                              # Define project name and language (C++)

set(CMAKE_CXX_STANDARD 17)                              # Use C++17 for std::optional, string_view, etc.
set(CMAKE_CXX_STANDARD_REQUIRED ON)                     # Fail build if compiler cannot do C++17

find_package(Boost REQUIRED COMPONENTS program_options)# Find Boost and specifically Program_options for CLI parsing
find_library(PCAP_LIB pcap REQUIRED)                    # Find libpcap library (packet capture backend)

add_executable(npa                                        # Create executable named "npa"
  src/main.cpp                                            # App orchestration (threads, timers, pipeline)
  src/config.cpp                                          # CLI parsing implementation
  src/util.cpp                                            # Byte helpers (endianness, ip formatting)
  src/pcap_capture.cpp                                    # Packet capture wrapper over libpcap
  src/parser.cpp                                          # Protocol parsing + HTTP Host extraction
  src/stats.cpp                                           # Counters + reporting
  src/pcap_writer.cpp                                     # PCAP export writer
  src/analytics.cpp                                       # Top talkers, top flows, top hosts
)

target_include_directories(npa PRIVATE include)         # Allow includes like "npa/parser.hpp" from include/
target_link_libraries(npa PRIVATE Boost::program_options ${PCAP_LIB}) # Link Boost.Program_options + libpcap
```

6.2 include/npa/config.hpp (CLI Configuration)

Purpose: Stores command-line settings in a single struct so all modules can use the same configuration.

Listing 2: include/npa/config.hpp (annotated)

```cpp
#pragma once                                          // Prevent multiple inclusion of this header
#include <string>                                     // Needed for std::string

namespace npa {                                       // Project namespace to avoid name conflicts

struct Config {                                       // Holds all CLI parameters (single source of truth)
  std::string iface;                                  // Network interface name (eth0, wlan0, etc.)
  std::string filter;                                 // BPF filter string (e.g., "tcp or udp")
  bool promisc = true;                                // Enable promiscuous mode to see more traffic
  int snaplen = 65535;                                // Max bytes captured per packet (full frame)
  int stats_every_sec = 1;                            // Print stats every N seconds
  bool verbose = false;                               // If true, print per-packet decoded output

  std::string pcap_out;                               // Output .pcap filename; empty means disabled
  int top_talkers = 5;                                // Top N talkers (0 disables)
  int top_flows = 5;                                  // Top N flows (0 disables)
  int top_hosts = 5;                                  // Top N HTTP hosts (0 disables)
};

Config parse_args(int argc, char** argv);             // Parse CLI into Config; exits on --help

} // namespace npa                                    // End namespace
```

6.3 src/config.cpp (CLI Parsing with Boost.Program_options)

Purpose: Converts `argv` into `Config`. This keeps main.cpp clean and professional.

Listing 3: src/config.cpp (annotated)

```cpp
#include "npa/config.hpp"                             // Import Config struct + parse_args declaration
#include <boost/program_options.hpp>                  // Boost CLI parsing library
#include <iostream>                                   // For printing help text

namespace npa {                                       // Keep everything under npa namespace

```

```
Config parse_args(int argc, char** argv) {                    // Entry function to parse command line
  namespace po = boost::program_options;                      // Alias for shorter code

  Config cfg;                                                 // Create config object to fill from CLI
  po::options_description desc("Network Packet Analyzer options"); // Help/usage text container

  desc.add_options()                                          // Define allowed CLI options
    ("help,h", "Show help")                                   // --help or -h prints help and exits
    ("iface,i", po::value<std::string>(&cfg.iface)->default_value("eth0"),
      "Interface to capture on")                              // Set interface name, default eth0
    ("filter,f", po::value<std::string>(&cfg.filter)->default_value("tcp or udp"),
      "BPF filter")                                           // Set BPF filter expression
    ("promisc", po::value<bool>(&cfg.promisc)->default_value(true),
      "Promiscuous mode (true/false)")                        // Toggle promiscuous capture
    ("snaplen", po::value<int>(&cfg.snaplen)->default_value(65535),
      "Snap length")                                          // How many bytes of each packet to capture
    ("stats-every", po::value<int>(&cfg.stats_every_sec)->default_value(1),
      "Print stats every N seconds")                          // Reporting interval
    ("verbose,v", po::bool_switch(&cfg.verbose),
      "Verbose output")                                       // -v prints each packet summary

    ("pcap-out", po::value<std::string>(&cfg.pcap_out)->default_value(""),
      "Write captured packets to this .pcap file")            // Enable PCAP output if non-empty
    ("top-talkers", po::value<int>(&cfg.top_talkers)->default_value(5),
      "Print top N SRC/DST talkers (0 disables)")             // Top talkers by bytes
    ("top-flows", po::value<int>(&cfg.top_flows)->default_value(5),
      "Print top N flows (0 disables)")                       // Top flows by bytes
    ("top-hosts", po::value<int>(&cfg.top_hosts)->default_value(5),
      "Print top N HTTP hosts (0 disables)");                 // Top HTTP Host headers by hits

  po::variables_map vm;                                       // Map to hold parsed values
  po::store(po::parse_command_line(argc, argv, desc), vm); // Parse
```

```
argv into vm
  po::notify(vm);                                    // Write parsed values into cfg fields

  if (vm.count("help")) {                            // If user requested help
    std::cout << desc << "\n";                       // Print help text
    std::exit(0);                                    // Exit program normally
  }

  if (cfg.stats_every_sec <= 0) cfg.stats_every_sec = 1;// Guard: avoid zero/negative timers
  if (cfg.snaplen <= 0) cfg.snaplen = 65535;         // Guard: invalid snaplen fallback
  if (cfg.top_talkers < 0) cfg.top_talkers = 0;      // Guard: negative disables
  if (cfg.top_flows < 0) cfg.top_flows = 0;          // Guard: negative disables
  if (cfg.top_hosts < 0) cfg.top_hosts = 0;          // Guard: negative disables

  return cfg;                                        // Return final validated configuration
}

} // namespace npa                                   // End namespace
```

6.4 include/npa/packet.hpp (PacketInfo Model)

Purpose: A single struct that represents **parsed** packet fields used by Stats and Analytics.

Listing 4: include/npa/packet.hpp (annotated)

```
#pragma once                                         // Prevent multiple inclusion
#include <cstdint>                                   // Fixed-width integer types (uint16_t etc.)
#include <string>                                    // std::string for IPs, HTTP fields

namespace npa {                                      // Project namespace

enum class L4Proto { TCP, UDP, OTHER };              // Layer-4 protocol type used in reporting

struct PacketInfo {                                  // Parsed packet metadata structure
  bool ipv4 = false;                                 // True if packet contains IPv4 header
  L4Proto l4 = L4Proto::OTHER;                        // Which L4 protocol was parsed
```

```

  std::string src_ip;                                   // Source IPv4 dotted string
  std::string dst_ip;                                   // Destination IPv4 dotted string

  uint16_t src_port = 0;                                // TCP/UDP source port (0 if unknown)
  uint16_t dst_port = 0;                                // TCP/UDP dest port (0 if unknown)

  uint16_t ip_total_len = 0;                            // IP total length field (helps debugging)
  uint8_t  ip_ttl = 0;                                  // IP TTL field (useful for diagnostics)

  bool http = false;                                    // True if an HTTP request was detected
  std::string http_method;                              // HTTP method e.g., GET, POST
  std::string http_uri;                                 // HTTP URI path
  std::string http_host;                                // HTTP Host header value

  uint32_t caplen = 0;                                  // Captured bytes in this packet (pcap caplen)
  uint32_t wirelen = 0;                                 // Actual on-wire size (pcap len)
};

} // namespace npa                                      // End namespace
```

6.5 include/npa/util.hpp and src/util.cpp (Byte Helpers)

Purpose: Networking protocols store integers in network byte order (big-endian). These helpers correctly read bytes and format IP addresses.

Listing 5: include/npa/util.hpp (annotated)

```
#pragma once                                            // Header include guard
#include <cstdint>                                      // For uint16_t/uint32_t
#include <string>                                       // For std::string

namespace npa {                                         // Project namespace

uint16_t read_be16(const uint8_t* p);                   // Read 16-bit big-endian from raw bytes
uint32_t read_be32(const uint8_t* p);                   // Read 32-bit big-endian from raw bytes
```

```
std::string ipv4_to_string(uint32_t be);                // Convert IPv4 (big-endian) to "x.x.x.x"

} // namespace npa
```

Listing 6: src/util.cpp (annotated)

```
#include "npa/util.hpp"                                 // Import util declarations
#include <sstream>                                      // For building string representation

namespace npa {

uint16_t read_be16(const uint8_t* p) {                  // Read 2 bytes big-endian
  return (uint16_t(p[0]) << 8) | uint16_t(p[1]);        // Combine bytes: p0 is high, p1 is low
}

uint32_t read_be32(const uint8_t* p) {                  // Read 4 bytes big-endian
  return (uint32_t(p[0]) << 24) |                       // Byte0 is highest-order
         (uint32_t(p[1]) << 16) |                       // Byte1
         (uint32_t(p[2]) << 8)  |                       // Byte2
          uint32_t(p[3]);                               // Byte3 is lowest-order
}

std::string ipv4_to_string(uint32_t be) {               // Convert IPv4 integer to dotted string
  uint8_t a = (be >> 24) & 0xFF;                        // Extract highest byte
  uint8_t b = (be >> 16) & 0xFF;                        // Extract next byte
  uint8_t c = (be >> 8)  & 0xFF;                        // Extract next byte
  uint8_t d = (be)       & 0xFF;                        // Extract lowest byte
  std::ostringstream oss;                                // Use stream to build string
  oss << int(a) << "." << int(b) << "." << int(c) << "." << int(d); // Build dotted IPv4
  return oss.str();                                      // Return final string
}

} // namespace npa
```

6.6 include/npa/pcap$_c$apture.hppandsrc/pcap$_c$apture.cpp(CaptureLayer)

Purpose: Wrap libpcap so main.cpp can run capture without touching C-style pcap API everywhere.

Listing 7: include/npa/pcap$_c$*apture.hpp*(*annotated*)

```cpp
#pragma once                                          // Prevent multiple inclusion
#include <cstdint>                                    // For uint32_t
#include <functional>                                 // For std::function callback
#include <string>                                     // For std::string iface/filter

namespace npa {

struct PcapPacket {                                   // Represents one captured packet delivered to callback
  const uint8_t* data;                                // Pointer to raw bytes (valid only during callback)
  uint32_t caplen;                                    // Bytes captured (may be <= wirelen)
  uint32_t wirelen;                                   // Actual length on wire
  const void* pcap_hdr;                               // Pointer to pcap_pkthdr (timestamp, lengths)
};

class PcapCapture {                                   // C++ wrapper around pcap_t*
public:
  using Callback = std::function<void(const PcapPacket&)>; // Callback signature for delivering packets

  PcapCapture() = default;                            // Default constructor
  ~PcapCapture();                                     // Destructor closes pcap handle safely

  void run(const std::string& iface,                  // Start capture on an interface
           const std::string& bpf_filter,             // Apply BPF filter if given
           bool promisc,                              // Promiscuous mode toggle
           int snaplen,                               // Snapshot length
           Callback cb);                              // Callback invoked per packet

  void stop();                                        // Stop capture loop cleanly

  void* native_handle() const { return handle_; }     // Expose raw pcap_t* for pcap_writer
private:
  void* handle_ = nullptr;                            // Stores pcap_t* as void* to avoid header leakage
  bool stop_ = false;                                 // Loop stop
```

```
    flag
};

} // namespace npa
```

Listing 8: src/pcap$_c$*apture.cpp*(*annotated*)

```
#include "npa/pcap_capture.hpp"                          // Our wrapper API
#include <pcap/pcap.h>                                   // libpcap C API
#include <stdexcept>                                     // For std::runtime_error
#include <string>                                        // For std::string operations

namespace npa {

struct DispatchUser {                                    // Struct passed to libpcap callback for context
  PcapCapture::Callback* cb;                             // Pointer to user callback stored outside C callback
};

static void pcap_dispatch_cb(u_char* user,               // libpcap callback signature
                             const struct pcap_pkthdr* h,
                             const u_char* bytes) {
  auto* u = reinterpret_cast<DispatchUser*>(user);       // Convert back to our context struct
  PcapPacket p{                                          // Build a PcapPacket wrapper
    reinterpret_cast<const uint8_t*>(bytes),             // Raw packet bytes
    static_cast<uint32_t>(h->caplen),                    // Captured length
    static_cast<uint32_t>(h->len),                       // On-wire length
    h                                                   // Store header pointer (timestamp/lengths)
  };
  (*(u->cb))(p);                                         // Call user callback with wrapped packet
}

PcapCapture::~PcapCapture() {                            // Destructor ensures handle is closed
  if (handle_) {                                         // If handle exists
    pcap_close(reinterpret_cast<pcap_t*>(handle_));      // Close capture session
    handle_ = nullptr;                                   // Reset pointer for safety
  }
}
```

```cpp

void PcapCapture::run(const std::string& iface,
                      const std::string& bpf_filter,
                      bool promisc,
                      int snaplen,
                      Callback cb) {
  char errbuf[PCAP_ERRBUF_SIZE]{0};                    // Buffer to store libpcap error messages

  pcap_t* p = pcap_open_live(                          // Open interface for live capture
      iface.c_str(),                                   // Interface name
      snaplen,                                         // Snapshot length per packet
      promisc ? 1 : 0,                                 // Promiscuous mode
      1000,                                            // Read timeout in ms
      errbuf);                                         // Output error buffer
  if (!p) throw std::runtime_error(std::string("pcap_open_live failed: ") + errbuf); // Fail if open fails

  handle_ = p;                                         // Store handle so writer can access it

  bpf_program fp{};                                    // Compiled BPF program container
  if (!bpf_filter.empty()) {                           // Only compile filter if user provided one
    if (pcap_compile(p, &fp, bpf_filter.c_str(), 1, PCAP_NETMASK_UNKNOWN) < 0) {
      throw std::runtime_error(std::string("pcap_compile failed: ") + pcap_geterr(p)); // Compilation error
    }
    if (pcap_setfilter(p, &fp) < 0) {                  // Install compiled filter into kernel capture path
      pcap_freecode(&fp);                              // Free compiled program memory
      throw std::runtime_error(std::string("pcap_setfilter failed: ") + pcap_geterr(p)); // Set filter error
    }
    pcap_freecode(&fp);                                // Free compiled filter after installing it
  }

  stop_ = false;                                       // Reset stop flag each time run starts
  DispatchUser u{&cb};                                 // Store pointer to callback for dispatch

  while (!stop_) {                                     // Capture loop continues until stop() called
```

```
    int rc = pcap_dispatch(p, 64, pcap_dispatch_cb, reinterpret_cast<u_char*>(&u)); // Process up to 64 packets
    if (rc < 0) break;                                  // Negative means error or breakloop triggered
  }
}

void PcapCapture::stop() {                              // Called by main thread to stop capture
  stop_ = true;                                         // Set stop flag
  if (handle_) {                                        // If capture handle exists
    pcap_breakloop(reinterpret_cast<pcap_t*>(handle_)); // Force pcap_dispatch to return early
  }
}

} // namespace npa
```

6.7 include/npa/pcap$_{w}$*riter.hppandsrc/pcap*$_{w}$*riter.cpp*(*PCAPExport*)

Purpose: Save captured packets to a `.pcap` file so results can be validated in Wireshark.

Listing 9: include/npa/pcap$_{w}$*riter.hpp*(*annotated*)

```
#pragma once                                            // Header guard
#include <pcap/pcap.h>                                  // Need pcap_t, pcap_pkthdr, pcap_dumper_t
#include <string>                                       // For filename storage

namespace npa {

class PcapWriter {                                      // Handles writing packets to a .pcap file
public:
  PcapWriter() = default;                               // Default constructor
  ~PcapWriter();                                        // Destructor closes file if open

  void open(pcap_t* handle, const std::string& filename); // Open dumper using capture handle
  void write(const pcap_pkthdr* hdr, const u_char* bytes); // Write one packet record
  void close();                                         // Close dumper handle

  bool enabled() const { return dumper_ != nullptr; }  // True if writer has a valid output file
private:
  pcap_dumper_t* dumper_ = nullptr;                     // libpcap dumper pointer (output stream)
};
```

```

} // namespace npa
```

Listing 10: src/pcap$_{w}$*riter.cpp*(*annotated*)

```
#include "npa/pcap_writer.hpp"                          // Import class declaration
#include <stdexcept>                                    // For exceptions on failure

namespace npa {

PcapWriter::~PcapWriter() { close(); }                  // Ensure resources are released automatically

void PcapWriter::open(pcap_t* handle, const std::string& filename) {
  if (filename.empty()) return;                         // If no filename, do nothing (feature disabled)
  if (!handle) throw std::runtime_error("PcapWriter::open: null pcap handle"); // Need handle to define link type

  dumper_ = pcap_dump_open(handle, filename.c_str());  // Open output file for writing pcap records
  if (!dumper_) {                                       // If open failed
    throw std::runtime_error("pcap_dump_open failed (check path/permissions)"); // Inform user
  }
}

void PcapWriter::write(const pcap_pkthdr* hdr, const u_char* bytes) {
  if (!dumper_ || !hdr || !bytes) return;               // Safety: only write if everything is valid
  pcap_dump(reinterpret_cast<u_char*>(dumper_), hdr, bytes); // Write packet header+data to file
}

void PcapWriter::close() {
  if (dumper_) {                                        // If file is open
    pcap_dump_close(dumper_);                           // Close output file and flush
    dumper_ = nullptr;                                  // Reset pointer to avoid double close
  }
}

} // namespace npa
```

6.8 include/npa/parser.hpp and src/parser.cpp (Protocol Parsing + HTTP Host Extraction)

Purpose: Convert raw bytes into meaningful protocol fields: src/dst IP, ports, protocol type, and (for plaintext HTTP) extract `Host` header.

Listing 11: include/npa/parser.hpp (annotated)

```cpp
#pragma once                                          // Header guard
#include "npa/packet.hpp"                             // PacketInfo definition
#include <cstdint>                                    // For uint32_t
#include <optional>                                   // For std::optional return type

namespace npa {

std::optional<PacketInfo> parse_packet(               // Parse a single packet frame (Ethernet->IP->TCP/UDP)
    const uint8_t* data,                              // Pointer to raw captured bytes
    uint32_t caplen,                                  // Captured length
    uint32_t wirelen);                                // On-wire length

} // namespace npa
```

Listing 12: src/parser.cpp (annotated)

```cpp
#include "npa/parser.hpp"                             // parse_packet declaration
#include "npa/util.hpp"                               // read_be16/read_be32/ipv4_to_string helpers
#include <algorithm>                                  // For std::min
#include <cctype>                                     // For tolower / isspace
#include <string_view>                                // For efficient parsing without copies

namespace npa {

static constexpr uint32_t ETH_HDR = 14;               // Ethernet header size (dest mac + src mac + type)
static constexpr uint16_t ETHERTYPE_IPV4 = 0x0800;     // Ethertype value for IPv4 frames

static bool starts_with(std::string_view s, std::string_view p) {
  return s.size() >= p.size() && s.substr(0, p.size()) == p; // Check prefix match (HTTP method detection)
}

static std::string trim(std::string s) {              // Remove leading/trailing whitespace (Host header cleanup)
  auto not_space = [](unsigned char c){ return !std::isspace(c); }; // Helper to detect non-whitespace
```

```
  while (!s.empty() && !not_space((unsigned char)s.front()))
      s.erase(s.begin()); // Trim left
  while (!s.empty() && !not_space((unsigned char)s.back()))
      s.pop_back();        // Trim right
  return s;                                          // Return
      trimmed string
}

static bool ieq_prefix(std::string_view s, std::string_view p) {
  if (s.size() < p.size()) return false;             // If s shorter
      than p, cannot match
  for (size_t i = 0; i < p.size(); i++) {            // Compare each
      char case-insensitively
    if (std::tolower((unsigned char)s[i]) != std::tolower((unsigned
        char)p[i])) return false;
  }
  return true;                                       // Prefix
      matches ignoring case
}

static void try_parse_http(PacketInfo& info, const uint8_t* payload,
    uint32_t payload_len) {
  if (!payload || payload_len < 16) return;          // HTTP request
      line needs at least some bytes

  uint32_t n = std::min<uint32_t>(payload_len, 4096);  // Limit scan
      for performance/safety
  std::string_view sv(reinterpret_cast<const char*>(payload), n); //
      View payload as string bytes

  const char* methods[] = {"GET ", "POST ", "HEAD ", "PUT ", "DELETE
      ", "OPTIONS ", "PATCH "}; // Supported methods
  bool is_http = false;                              // Tracks
      whether payload looks like HTTP
  std::string method;                                // Will store
      method if detected

  for (auto m : methods) {                           // Try each
      method prefix
    if (starts_with(sv, m)) {                        // If payload
        begins with "GET " etc.
      is_http = true;                                // Mark it as
          HTTP
      method = std::string(m);                       // Copy method
          text including trailing space
      method.pop_back();                             // Remove the
          trailing space => "GET"
      break;                                         // Stop once
          matched
    }
  }
  if (!is_http) return;                              // Not an HTTP
      request; exit

```

```
auto line_end = sv.find("\r\n");                    // Find end of request line
if (line_end == std::string_view::npos) return;     // If no CRLF, header incomplete

std::string_view line = sv.substr(0, line_end);     // The request line: "GET / HTTP/1.1"
size_t sp1 = line.find(' ');                        // First space after method
if (sp1 == std::string_view::npos) return;          // Invalid request line
size_t sp2 = line.find(' ', sp1 + 1);               // Second space after URI
if (sp2 == std::string_view::npos) return;          // Invalid request line

std::string uri(line.substr(sp1 + 1, sp2 - (sp1 + 1))); // Extract URI portion

std::string host;                                   // Will store Host header if found
size_t pos = line_end + 2;                          // Headers start after CRLF

while (pos < sv.size()) {                           // Iterate header lines
  size_t next = sv.find("\r\n", pos);               // Find next header line end
  if (next == std::string_view::npos) break;        // If not found, stop parsing
  if (next == pos) break;                           // Blank line => end of headers

  std::string_view hline = sv.substr(pos, next - pos); // Extract one header line
  if (ieq_prefix(hline, "Host:")) {                 // If this is Host header (case-insensitive)
    std::string v(hline.substr(5));                 // Get everything after "Host:"
    host = trim(v);                                 // Trim whitespace
    break;                                          // Stop once Host found
  }
  pos = next + 2;                                   // Move to next header line
}

info.http = true;                                   // Mark packet as HTTP
info.http_method = method;                          // Store method
info.http_uri = uri;                                // Store URI
info.http_host = host;                              // Store Host (may be empty if missing)
```

```
}

std::optional<PacketInfo> parse_packet(const uint8_t* data, uint32_t caplen, uint32_t wirelen) {
  PacketInfo info;                                    // Create PacketInfo result
  info.caplen = caplen;                               // Store captured length for reporting
  info.wirelen = wirelen;                             // Store on-wire length for bandwidth stats

  if (!data || caplen < ETH_HDR) return std::nullopt;  // Need at least Ethernet header bytes

  uint16_t etherType = read_be16(data + 12);          // Ethertype stored at bytes 12-13
  if (etherType != ETHERTYPE_IPV4) return std::nullopt;// Only handle IPv4 frames in this project

  const uint8_t* ip = data + ETH_HDR;                 // IP header begins after Ethernet header
  uint32_t ip_avail = caplen - ETH_HDR;               // Bytes available after Ethernet header
  if (ip_avail < 20) return std::nullopt;             // Minimum IPv4 header is 20 bytes

  uint8_t ver_ihl = ip[0];                            // First byte has version + IHL
  uint8_t ver = ver_ihl >> 4;                         // High 4 bits = version
  uint8_t ihl = (ver_ihl & 0x0F) * 4;                 // Low 4 bits = header length in 32-bit words => bytes
  if (ver != 4 || ihl < 20) return std::nullopt;      // Must be IPv4 + valid header length
  if (ip_avail < ihl) return std::nullopt;            // Ensure captured bytes include whole IP header

  info.ipv4 = true;                                   // Mark IPv4 present
  info.ip_total_len = read_be16(ip + 2);              // Total length field in IP header
  info.ip_ttl = ip[8];                                // TTL field
  uint8_t proto = ip[9];                              // Protocol field (6 TCP, 17 UDP)

  uint32_t src_be = read_be32(ip + 12);               // Source IP address bytes
  uint32_t dst_be = read_be32(ip + 16);               // Destination IP address bytes
  info.src_ip = ipv4_to_string(src_be);               // Convert src IP to dotted string
  info.dst_ip = ipv4_to_string(dst_be);               // Convert dst IP to dotted string

```

```
const uint8_t* l4 = ip + ihl;                          // L4 header starts after IP header
uint32_t l4_avail = ip_avail - ihl;                    // Remaining bytes after IP header

if (proto == 6) {                                      // TCP protocol
  if (l4_avail < 20) return info;                      // Need minimum TCP header
  info.l4 = L4Proto::TCP;                              // Mark protocol as TCP

  info.src_port = read_be16(l4 + 0);                   // TCP src port
  info.dst_port = read_be16(l4 + 2);                   // TCP dst port

  uint8_t data_offset = (l4[12] >> 4) * 4;             // TCP header length (data offset) in bytes
  if (data_offset < 20 || l4_avail < data_offset) return info; // Validate header length

  const uint8_t* payload = l4 + data_offset;           // Payload begins after TCP header
  uint32_t payload_len = l4_avail - data_offset;       // Payload length

  if (info.src_port == 80 || info.dst_port == 80 || info.src_port == 8080 || info.dst_port == 8080) {
    try_parse_http(info, payload, payload_len);        // Parse HTTP if common HTTP ports
  } else {
    try_parse_http(info, payload, payload_len);        // Heuristic: try anyway for labs/non-standard ports
    if (!info.http) {                                  // If not HTTP, clear fields to avoid confusion
      info.http_method.clear();
      info.http_uri.clear();
      info.http_host.clear();
    }
  }
  return info;                                         // Return parsed TCP packet

} else if (proto == 17) {                              // UDP protocol
  if (l4_avail < 8) return info;                       // UDP header is 8 bytes minimum
  info.l4 = L4Proto::UDP;                              // Mark as UDP
  info.src_port = read_be16(l4 + 0);                   // UDP src port
  info.dst_port = read_be16(l4 + 2);                   // UDP dst port
  return info;                                         // Return parsed UDP packet

} else {                                               // Other protocols (ICMP etc.)
  info.l4 = L4Proto::OTHER;                            // Mark unknown/other
```

```
    return info;                                        // Return partial information (IPv4 addresses still useful)
  }
}

} // namespace npa
```

6.9 include/npa/analytics.hpp and src/analytics.cpp (Flows, Talkers, HTTP Hosts)

Purpose: Provide Wireshark-like intelligence: which IPs talk most, which flows dominate bandwidth, and which HTTP hosts are accessed.

Listing 13: include/npa/analytics.hpp (annotated)

```
#pragma once                                            // Header guard
#include "npa/packet.hpp"                               // PacketInfo input type
#include <cstdint>                                      // uint64_t etc.
#include <string>                                       // std::string keys
#include <unordered_map>                                // hash maps for counters
#include <vector>                                       // vector outputs for top-N lists

namespace npa {

struct TalkerEntry {                                    // One entry in top-talkers list
  std::string ip;                                       // IP address
  uint64_t bytes = 0;                                   // Total bytes seen for that IP
};

struct HostEntry {                                      // One entry in top-hosts list
  std::string host;                                     // HTTP Host value
  uint64_t hits = 0;                                    // Number of requests seen for that host
};

enum class FlowProto : uint8_t { TCP = 6, UDP = 17, OTHER = 0 }; // Flow protocol tag

struct Endpoint {                                       // One endpoint in a flow key
  std::string ip;                                       // IP address
  uint16_t port = 0;                                    // Port number
  bool operator<(const Endpoint& o) const {             // Used for canonicalization ordering
    if (ip != o.ip) return ip < o.ip;                   // Compare by IP first
```

```
    return port < o.port;                                   // Tie-break by port
  }
};

struct FlowKey {                                            // Canonical flow identifier
  FlowProto proto;                                          // TCP/UDP/OTHER
  Endpoint a;                                               // Endpoint A (ordered)
  Endpoint b;                                               // Endpoint B (ordered)
  bool operator==(const FlowKey& o) const {                 // Equality for unordered_map key
    return proto == o.proto &&
           a.ip == o.a.ip && a.port == o.a.port &&
           b.ip == o.b.ip && b.port == o.b.port;
  }
};

struct FlowKeyHash { std::size_t operator()(const FlowKey& k) const; }; // Hash function for FlowKey

struct FlowEntry {                                          // Stored stats per flow
  FlowKey key;                                              // Flow identifier
  uint64_t packets = 0;                                     // Packets count
  uint64_t bytes = 0;                                       // Bytes count
  uint64_t first_seen_ms = 0;                               // First time observed (ms)
  uint64_t last_seen_ms = 0;                                // Last time observed (ms)
};

class Analytics {                                           // Main analytics engine
public:
  void ingest(const PacketInfo& p, uint64_t ts_ms);      // Update analytics with a parsed packet

  std::vector<TalkerEntry> top_src_talkers(int N) const;// Top N source IPs by bytes
  std::vector<TalkerEntry> top_dst_talkers(int N) const;// Top N destination IPs by bytes
  std::vector<FlowEntry> top_flows(int N) const;         // Top N flows by bytes
  std::vector<HostEntry> top_http_hosts(int N) const;    // Top N HTTP hosts by hits

private:
  std::unordered_map<std::string, uint64_t> src_bytes_; // Map srcIP -> bytes
  std::unordered_map<std::string, uint64_t> dst_bytes_; // Map dstIP
```

```
    -> bytes
  std::unordered_map<FlowKey, FlowEntry, FlowKeyHash> flows_; // Map flowKey -> flowStats
  std::unordered_map<std::string, uint64_t> http_host_hits_;  // Map host -> hits

  static FlowKey make_flow_key(const PacketInfo& p);    // Build canonical key from packet fields
};

} // namespace npa
```

Listing 14: src/analytics.cpp (annotated)

```
#include "npa/analytics.hpp"                          // Analytics declarations
#include <algorithm>                                  // sort, nth_element
#include <functional>                                 // std::hash

namespace npa {

static std::size_t hcombine(std::size_t h1, std::size_t h2) {
  return h1 ^ (h2 + 0x9e3779b97f4a7c15ULL + (h1 << 6) + (h1 >> 2)); // Combine hashes (common technique)
}

std::size_t FlowKeyHash::operator()(const FlowKey& k) const {
  std::hash<std::string> hs;                          // Hash string values (IPs)
  std::hash<uint16_t> hp;                             // Hash ports
  std::hash<uint8_t> hb;                              // Hash proto tag
  std::size_t h = 0;                                  // Start hash accumulator
  h = hcombine(h, hb(static_cast<uint8_t>(k.proto)));   // Add protocol to hash
  h = hcombine(h, hs(k.a.ip));                        // Add endpoint A IP
  h = hcombine(h, hp(k.a.port));                      // Add endpoint A port
  h = hcombine(h, hs(k.b.ip));                        // Add endpoint B IP
  h = hcombine(h, hp(k.b.port));                      // Add endpoint B port
  return h;                                           // Return final hash value
}

FlowKey Analytics::make_flow_key(const PacketInfo& p) {
  FlowProto fp = FlowProto::OTHER;                    // Default unknown
  if (p.l4 == L4Proto::TCP) fp = FlowProto::TCP;      // Map PacketInfo l4 to FlowProto tag
```

```
  else if (p.l4 == L4Proto::UDP) fp = FlowProto::UDP;  // UDP mapping

  Endpoint e1{p.src_ip, p.src_port};                   // Endpoint from src fields
  Endpoint e2{p.dst_ip, p.dst_port};                   // Endpoint from dst fields
  if (e2 < e1) std::swap(e1, e2);                      // Canonicalize: order endpoints so A<->B is same flow
  return FlowKey{fp, e1, e2};                          // Return final canonical key
}

static std::vector<TalkerEntry> topN_talkers(const std::unordered_map<std::string, uint64_t>& mp, int N) {
  if (N <= 0) return {};                               // If disabled, return empty
  std::vector<TalkerEntry> v;                          // Convert map to vector for sorting
  v.reserve(mp.size());                                // Reserve for performance
  for (const auto& kv : mp) v.push_back({kv.first, kv.second}); // Move each map entry to vector
  if ((int)v.size() > N) {                             // If more than N entries, partially select top N
    std::nth_element(v.begin(), v.begin() + N, v.end(), // nth_element is O(n) average
      [](const TalkerEntry& a, const TalkerEntry& b){ return a.bytes > b.bytes; });
    v.resize(N);                                       // Keep only top N candidates
  }
  std::sort(v.begin(), v.end(),                        // Sort top N for final ordering
    [](auto& a, auto& b){ return a.bytes > b.bytes; });
  return v;                                            // Return sorted top N
}

static std::vector<HostEntry> topN_hosts(const std::unordered_map<std::string, uint64_t>& mp, int N) {
  if (N <= 0) return {};                               // Disabled => empty
  std::vector<HostEntry> v;                            // Convert map to vector
  v.reserve(mp.size());                                // Reserve
  for (const auto& kv : mp) v.push_back({kv.first, kv.second}); // host -> hits
  if ((int)v.size() > N) {                             // Partially select top N
    std::nth_element(v.begin(), v.begin() + N, v.end(),
      [](const HostEntry& a, const HostEntry& b){ return a.hits > b.hits; });
    v.resize(N);                                       // Keep top N
  }
```

```
  std::sort(v.begin(), v.end(),                          // Sort descending
    [](auto& a, auto& b){ return a.hits > b.hits; });
  return v;                                              // Return list
}

void Analytics::ingest(const PacketInfo& p, uint64_t ts_ms) {
  if (!p.ipv4) return;                                   // Only analyze IPv4 in this project

  src_bytes_[p.src_ip] += p.wirelen;                     // Add bytes to source talker map
  dst_bytes_[p.dst_ip] += p.wirelen;                     // Add bytes to destination talker map

  FlowKey key = make_flow_key(p);                        // Build canonical flow key
  auto it = flows_.find(key);                            // Check if flow already exists
  if (it == flows_.end()) {                              // If new flow
    FlowEntry fe;                                        // Create new flow entry
    fe.key = key;                                        // Store flow key
    fe.packets = 1;                                      // First packet
    fe.bytes = p.wirelen;                                // First bytes count
    fe.first_seen_ms = ts_ms;                            // First time seen
    fe.last_seen_ms = ts_ms;                             // Last time seen initially same
    flows_.emplace(key, std::move(fe));                  // Insert into map
  } else {                                               // Existing flow
    it->second.packets += 1;                             // Increment packet count
    it->second.bytes += p.wirelen;                       // Add bytes
    it->second.last_seen_ms = ts_ms;                     // Update last seen timestamp
  }

  if (p.http && !p.http_host.empty()) {                  // If HTTP detected and host exists
    http_host_hits_[p.http_host] += 1;                   // Increment host popularity counter
  }
}

std::vector<TalkerEntry> Analytics::top_src_talkers(int N) const {
  return topN_talkers(src_bytes_, N);                    // Delegate to helper for top N selection
}

std::vector<TalkerEntry> Analytics::top_dst_talkers(int N) const {
```

```
  return topN_talkers(dst_bytes_, N);                    // Delegate to helper
}

std::vector<FlowEntry> Analytics::top_flows(int N) const {
  if (N <= 0) return {};                                 // Disabled => empty
  std::vector<FlowEntry> v;                              // Convert map to vector
  v.reserve(flows_.size());                              // Reserve
  for (const auto& kv : flows_) v.push_back(kv.second);// Copy each flow entry

  if ((int)v.size() > N) {                               // Keep only top N by bytes efficiently
    std::nth_element(v.begin(), v.begin() + N, v.end(),
      [](const FlowEntry& a, const FlowEntry& b){ return a.bytes > b.bytes; });
    v.resize(N);                                         // Reduce vector
  }
  std::sort(v.begin(), v.end(),                          // Sort final top N list
    [](auto& a, auto& b){ return a.bytes > b.bytes; });
  return v;                                              // Return top flows
}

std::vector<HostEntry> Analytics::top_http_hosts(int N) const {
  return topN_hosts(http_host_hits_, N);                 // Delegate to helper
}

} // namespace npa
```

6.10 include/npa/stats.hpp and src/stats.cpp (Counters + Reporting)

Purpose: Centralized counters and formatted reporting. Also integrates Analytics.

Listing 15: include/npa/stats.hpp (annotated)

```
#pragma once                                             // Header guard
#include "npa/packet.hpp"                                // PacketInfo type
#include "npa/analytics.hpp"                             // Analytics engine
#include <atomic>                                        // Atomic counters for thread-safe increments
#include <cstdint>                                       // uint64_t
#include <mutex>                                         // Mutex for protecting Analytics maps
#include <string>                                        // Return formatted string reports

namespace npa {
```

```

class Stats {                                          // Holds counters + Analytics under one roof
public:
  void ingest(const PacketInfo& p, uint64_t ts_ms);     // Called per parsed packet
  std::string report_and_reset_interval(int top_talkers,// Build report string; reset interval counters
                                        int top_flows,
                                        int top_hosts);

private:
  std::atomic<uint64_t> total_packets_{0};              // Total packet counter since start
  std::atomic<uint64_t> total_bytes_{0};                // Total bytes since start

  std::atomic<uint64_t> tcp_packets_{0};                // Count of TCP packets
  std::atomic<uint64_t> udp_packets_{0};                // Count of UDP packets
  std::atomic<uint64_t> other_packets_{0};              // Count of other L4 protocols

  std::atomic<uint64_t> interval_packets_{0};           // Packets in current interval window
  std::atomic<uint64_t> interval_bytes_{0};             // Bytes in current interval window

  std::atomic<uint64_t> http_requests_{0};              // Total HTTP requests detected

  mutable std::mutex mx_;                               // Mutex for Analytics (unordered_map not thread-safe)
  Analytics analytics_;                                 // Analytics engine instance
};

} // namespace npa
```

Listing 16: src/stats.cpp (annotated)

```
#include "npa/stats.hpp"                                // Stats declaration
#include <sstream>                                      // Build report string

namespace npa {

void Stats::ingest(const PacketInfo& p, uint64_t ts_ms) {
  total_packets_.fetch_add(1, std::memory_order_relaxed); // Atomically increment total packets
  total_bytes_.fetch_add(p.wirelen, std::memory_order_relaxed); // Atomically add bytes
```

```

  interval_packets_.fetch_add(1, std::memory_order_relaxed); // Interval packets++
  interval_bytes_.fetch_add(p.wirelen, std::memory_order_relaxed); // Interval bytes +=

  switch (p.l4) {                                          // Update protocol counters based on packet L4
    case L4Proto::TCP: tcp_packets_.fetch_add(1, std::memory_order_relaxed); break; // TCP++
    case L4Proto::UDP: udp_packets_.fetch_add(1, std::memory_order_relaxed); break; // UDP++
    default: other_packets_.fetch_add(1, std::memory_order_relaxed); break;          // OTHER++
  }

  if (p.http) http_requests_.fetch_add(1, std::memory_order_relaxed); // HTTP counter increments if detected

  {                                                        // Begin critical section for Analytics updates
    std::lock_guard<std::mutex> lk(mx_);                   // Lock mutex to protect maps inside Analytics
    analytics_.ingest(p, ts_ms);                           // Update analytics with this packet
  }                                                        // Mutex unlocks here
}

std::string Stats::report_and_reset_interval(int top_talkers, int top_flows, int top_hosts) {
  uint64_t ip = interval_packets_.exchange(0, std::memory_order_relaxed); // Read+reset interval packets
  uint64_t ib = interval_bytes_.exchange(0, std::memory_order_relaxed);   // Read+reset interval bytes

  uint64_t tp = total_packets_.load(std::memory_order_relaxed);              // Read total packets
  uint64_t tb = total_bytes_.load(std::memory_order_relaxed);                // Read total bytes

  uint64_t tcp = tcp_packets_.load(std::memory_order_relaxed);               // Read TCP count
  uint64_t udp = udp_packets_.load(std::memory_order_relaxed);               // Read UDP count
  uint64_t oth = other_packets_.load(std::memory_order_relaxed);             // Read OTHER count
  uint64_t http = http_requests_.load(std::memory_order_relaxed);            // Read HTTP requests count

  std::ostringstream oss;                                  // Build formatted output into a string
  oss << "Interval: packets=" << ip << ", bytes=" << ib // Interval counters
```

```
      << " | Total: packets=" << tp << ", bytes=" << tb // Total counters
      << " | proto: TCP=" << tcp << " UDP=" << udp << " OTHER=" << oth // Protocol breakdown
      << " | HTTP(req)=" << http;                       // HTTP request counter

  std::lock_guard<std::mutex> lk(mx_);                  // Lock Analytics for consistent top-N reads

  if (top_talkers > 0) {                                // If talkers enabled
    auto topSrc = analytics_.top_src_talkers(top_talkers); // Get top src talkers
    auto topDst = analytics_.top_dst_talkers(top_talkers); // Get top dst talkers
    oss << "\n  Top SRC talkers(bytes): ";              // Print src talkers line
    for (const auto& t : topSrc) oss << t.ip << "=" << t.bytes << " "; // Append each talker
    oss << "\n  Top DST talkers(bytes): ";              // Print dst talkers line
    for (const auto& t : topDst) oss << t.ip << "=" << t.bytes << " "; // Append each talker
  }

  if (top_flows > 0) {                                  // If flows enabled
    auto flows = analytics_.top_flows(top_flows);       // Get top flows
    oss << "\n  Top Flows(bytes):";                     // Header for flows list
    for (const auto& f : flows) {                       // Iterate flows
      std::string proto = "OTH";                        // Default proto label
      if (f.key.proto == FlowProto::TCP) proto = "TCP";// Map FlowProto to label
      else if (f.key.proto == FlowProto::UDP) proto = "UDP";
      oss << "\n    " << proto << " "                  // Indented flow line
          << f.key.a.ip << ":" << f.key.a.port          // Endpoint A
          << " <-> "                                    // Bidirectional conversation arrow
          << f.key.b.ip << ":" << f.key.b.port          // Endpoint B
          << " bytes=" << f.bytes                       // Flow bytes
          << " pkts=" << f.packets;                     // Flow packets
    }
  }

  if (top_hosts > 0) {                                  // If HTTP host list enabled
    auto hosts = analytics_.top_http_hosts(top_hosts); // Get top HTTP hosts
    oss << "\n  Top HTTP Hosts(hits): ";                // Host list line
```

```
    for (const auto& h : hosts) oss << h.host << "=" << h.hits << " "; // Append each host
  }

  return oss.str();                                    // Return final report string
}

} // namespace npa
```

6.11 src/main.cpp (Orchestration, Threads, Timers, Pipeline)

Purpose: The control center that wires everything together: capture, parsing, analytics, periodic reporting, and shutdown.

Listing 17: src/main.cpp (annotated)

```
#include "npa/config.hpp"                              // CLI parsing + Config struct
#include "npa/pcap_capture.hpp"                        // Packet capture wrapper
#include "npa/pcap_writer.hpp"                         // PCAP export writer
#include "npa/parser.hpp"                              // parse_packet()
#include "npa/stats.hpp"                               // Stats + Analytics integration

#include <boost/asio.hpp>                              // Boost.Asio for async task posting + timers
#include <chrono>                                      // For timestamps/timers
#include <iostream>                                    // Console I/O
#include <thread>                                      // std::thread
#include <vector>                                      // std::vector of threads

static uint64_t now_ms() {                             // Helper: current monotonic time in milliseconds
  using namespace std::chrono;                         // Bring chrono names into scope
  return duration_cast<milliseconds>(steady_clock::now().time_since_epoch()).count(); // Monotonic time
}

int main(int argc, char** argv) {                      // Program entry point
  using namespace npa;                                 // Use project namespace for convenience

  Config cfg = parse_args(argc, argv);                 // Parse CLI and build configuration

```

```
boost::asio::io_context io;                                   // Asio event loop used for worker tasks and timers
Stats stats;                                                  // Stats object storing counters + analytics

boost::asio::steady_timer timer(io);                          // Timer used for periodic report printing
std::function<void()> tick;                                   // Function object to schedule recurring timer events
tick = [&]() {                                                // Define recurring timer behavior
  timer.expires_after(std::chrono::seconds(cfg.stats_every_sec)); // Next trigger after N seconds
  timer.async_wait([&](const boost::system::error_code& ec) {     // Non-blocking wait callback
    if (!ec) {                                                // If timer not cancelled
      std::cout << stats.report_and_reset_interval(cfg.top_talkers, cfg.top_flows, cfg.top_hosts) << "\n"; // Print report
      tick();                                                 // Reschedule next tick (recursion via function object)
    }
  });
};
tick();                                                       // Start the periodic reporting loop

const int workers = std::max(1u, std::thread::hardware_concurrency()); // Worker thread count
std::vector<std::thread> pool;                                // Worker threads container
pool.reserve(workers);                                        // Reserve memory to avoid reallocations
for (int i = 0; i < workers; i++) {                           // Spawn worker threads
  pool.emplace_back([&]() { io.run(); });                     // Each thread runs the Asio event loop
}

PcapCapture cap;                                              // Capture engine instance
PcapWriter writer;                                            // PCAP file writer instance

std::thread cap_thread([&]() {                                // Run capture loop on a dedicated thread
  try {                                                       // Capture may throw exceptions; protect thread
    cap.run(cfg.iface, cfg.filter, cfg.promisc, cfg.snaplen, // Start capture with config parameters
      [&](const PcapPacket& p) {                              // Callback invoked for each captured packet
        if (!cfg.pcap_out.empty() && !writer.enabled() && cap.native_handle() != nullptr) { // Lazy open pcap writer
```

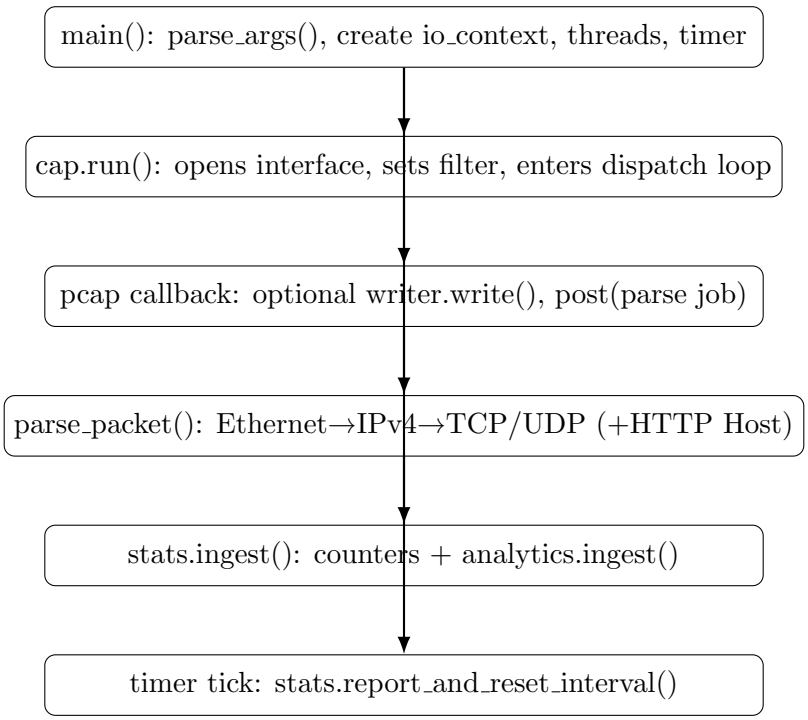
```
            try {
              writer.open(reinterpret_cast<pcap_t*>(cap.native_handle()), cfg.pcap_out);      // Open .pcap dumper
              std::cerr << "PCAP output enabled: " << cfg.pcap_out << "\n";                   // Confirm to user
            } catch (const std::exception& e) {
              std::cerr << "PCAP output error: " << e.what() << "\n";                          // Report writer error
            }
          }

          if (writer.enabled()) {                                  // If pcap writer is active
            writer.write(reinterpret_cast<const pcap_pkthdr*>(p.pcap_hdr),                    // Write header
                         reinterpret_cast<const u_char*>(p.data));                             // Write bytes
          }

          auto buf = std::make_shared<std::vector<uint8_t>>(p.data, p.data + p.caplen);       // Copy bytes for async parsing safety

          boost::asio::post(io, [buf, p, &stats, cfg]() {                                     // Post parsing job to worker pool
            auto parsed = parse_packet(buf->data(), p.caplen, p.wirelen);                     // Parse packet bytes
            if (!parsed) return;                                                              // Ignore if not IPv4 or parse failed

            stats.ingest(*parsed, now_ms());                                                  // Update stats + analytics

            if (cfg.verbose) {                                                                // If verbose enabled, print per packet
              const auto& x = *parsed;                                                        // Reference parsed packet
              std::cout
                << (x.l4 == L4Proto::TCP ? "TCP" : (x.l4 == L4Proto::UDP ? "UDP" : "OTH"))     // Protocol label
                << " " << x.src_ip << ":" << x.src_port                                       // Source endpoint
                << " -> " << x.dst_ip << ":" << x.dst_port                                    // Destination endpoint
                << " len=" << x.wirelen;
```

```
                    // Packet size

            if (x.http) {

                // If HTTP request detected
              std::cout << " | HTTP " << x.http_method << " " << x.http_uri;                    // Print method+uri
              if (!x.http_host.empty()) std::cout << " Host=" << x.http_host;                    // Print host if available
            }
            std::cout << "\n";

                // End line
          }
        });
      }
    );
  } catch (const std::exception& e) {
    std::cerr << "Capture error: " << e.what() << "\n";                                          // Report capture errors
  }
});

std::cout << "Capturing on " << cfg.iface << " with filter: " << cfg.filter << "\n";        // Startup message
if (!cfg.pcap_out.empty()) std::cout << "PCAP output: " << cfg.pcap_out << "\n";             // PCAP message if enabled
std::cout << "Press ENTER to stop...\n";                                                        // User stop instruction
std::cin.get();                                                                                 // Wait for ENTER

cap.stop();                                                                                     // Stop capture loop
io.stop();                                                                                      // Stop Asio workers

cap_thread.join();                                                                              // Wait for capture thread to finish
writer.close();                                                                                 // Close pcap output file
for (auto& t : pool) t.join();                                                                  // Join worker threads

return 0;
```

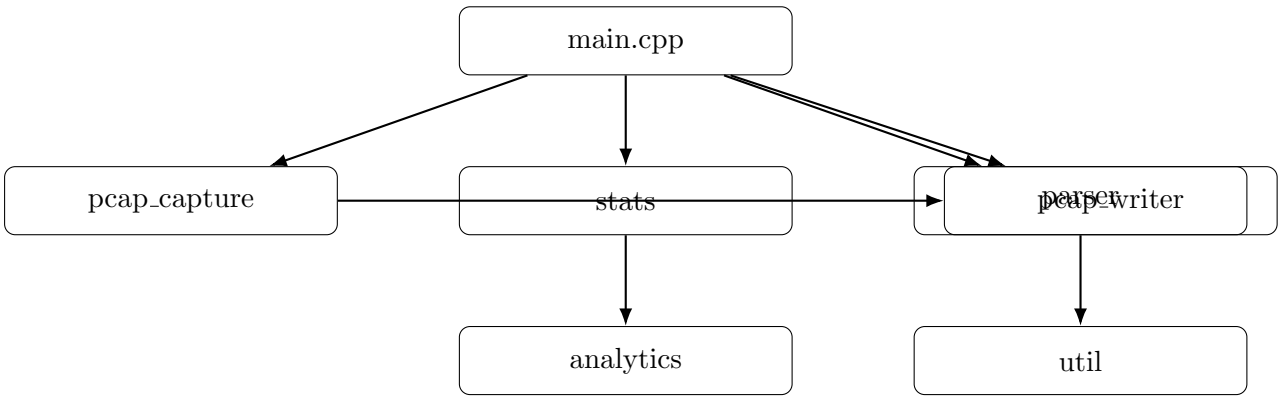
```
    // Successful exit
}
```

7 Extra Diagrams for Interviews

7.1 Call Graph



7.2 Dependency Map



8 Notes and Limitations (Honest Engineering)

- HTTP Host extraction works only for **plaintext HTTP** (common on port 80). HTTPS encrypts headers.
- TCP stream reassembly is not implemented. If a request spans multiple TCP segments, Host might not be extracted.

- IPv6 is not implemented (can be extended in parser.cpp).
- Flow eviction/timeouts are not implemented (can be added to Analytics for long-running capture).

9 Future Work (Resume-Heavy Extensions)

- TLS SNI extraction (HTTPS host visibility)
- IPv6 parsing
- JSON export for dashboards
- Flow eviction + idle timeout
- ncurses terminal UI
- PCAP replay mode (offline analysis)

10 Conclusion

This report provided:

- complete folder + file mapping,
- complete source code for each file as code blocks,
- line-by-line explanation via inline comments,
- system architecture diagrams using TikZ,
- interview-ready reasoning and technical depth.